Conferencia 7: Programación Declarativa

Lic. Hiram Borbolla Hernández

Tema: Introducción a la Programación Funcional. Definición de Funciones. Tipos y clases de tipos.

La *programación funcional* (PF) es uno de los paradigmas representativos de la programación declarativa. De manera general, se caracteriza por usar el lenguaje de las funciones matemáticas para escribir un programa y como mecanismo computacional la reducción de expresiones (funcionales) para obtener un *valor*.

Como en la PL, en la PF la tarea de programar consiste en especificar el *qué se desea computar* y no el *cómo se deben realizar los cómputos*. El programador se centra en la descripción del problema dejando a un lado cuestiones de bajo nivel como la organización de la secuencia de cómputo en pequeños pasos, el número de llamadas a funciones, la gestión de la memoria, etc.

A diferencia de los lenguajes imperativos en los que se expresa cómo hacer un cálculo mediante una secuencia de órdenes a la máquina, en los lenguajes funcionales se describe el problema que se desea resolver mediante funciones matemáticas. Por ejemplo, la función factorial, imperativamente, se definiría como la secuencia de órdenes: (1) restar al valor de entrada, digamos n, el número 1, (2) asignar el valor obtenido a la variable m, (2) calcular el factorial de m, (3) devolver el producto entre n y el factorial de m. En cambio, funcionalmente se definiría tal y como se define el factorial de un número: "*el factorial de* n *es* n *por el factorial de* n-1". Utilizando la sintaxis de un lenguaje funcional (Haskell) su definición sería mediante la ecuación:

```
factorial n = n * factorial (n-1)
```

Los programas escritos en un lenguaje funcional están constituidos únicamente por definiciones de funciones, entendiendo éstas no como subprogramas clásicos de un lenguaje imperativo, sino como funciones puramente matemáticas, en las que se verifican ciertas propiedades como la *transparencia referencial* (el resultado de una función depende únicamente del valor de sus argumentos), y por tanto, la ausencia de *efectos colaterales*.

El programador puede concentrarse en definir un conjunto de funciones sin preocuparse de los métodos de evaluación que posteriormente utilice el sistema. Este modelo deja mayor libertad al sistema de evaluación para incorporar pasos intermedios de transformación de código y paralelización, ya que las funciones no tienen efectos colaterales y no dependen de una arquitectura concreta.

En los lenguajes funcionales no existe el concepto de asignación de variables (cuando una variable toma un valor esta no puede ser luego modificada) y las construcciones estructuradas como la secuencia o la iteración (lo que obliga en la práctica a que todas las repeticiones de instrucciones se lleven a cabo por medio de funciones recursivas).

Fundamentos

La PF constituye una implementación computacional del *Cálculo Lambda* (λ-Calculus), un sistema formal desarrollado en 1930 por el lógico matemático Alonzo Church, como apoyo a sus estudios sobre computabilidad, basado en la definición de funciones que toman otras funciones como argumento y que devuelven funciones como resultado. Básicamente, el cálculo lambda consiste de un esquema simple de definición de funciones, además de una regla de sustitución de variables:

- Definición: consiste en definir funciones de un solo argumento y con un cuerpo específico, denotado por $\lambda x.B$, donde x representa el parámetro o argumento formal y B representa el cuerpo de la función.
  Dado que no es necesario, y relevante, nombrar las funciones para definirlas y utilizarlas, estas son expresadas anónimamente con un único símbolo, la letra griega λ (*lambda*). La aplicación de una función a sus argumentos se representa mediante la notación $\lambda x.B$ que puede entenderse como $f(x) = B$.

- Reducción: consiste en aplicar alguna de las funciones creadas, sobre un argumento real (a) luego de sustituir las ocurrencias del argumento formal (x), que aparezcan en el cuerpo (B) de la función, con el argumento (a).

Por ejemplo, la expresión lambda $(\lambda x.(x+5))3$, indica que en la expresión $x+5$, se debe sustituir el valor de x por 3, lo cual daría como resultado el valor 8. Cuando ya no es posible reducir una función, se dice que ésta se encuentra en su estado normal, o sea que se ha encontrado el valor de la función, que dependerá únicamente de sus argumentos.

Otra característica del cálculo lambda es que las funciones son definidas con un solo argumento. Esto se debe a que cualquier función con más de un argumento puede transformarse en otra que acepte un solo argumento. Por ejemplo, un función de dos argumentos como entrada puede ser transformada a una función que acepta el primer argumento y devuelve otra función que acepta el segundo, es decir, las funciones que toman más de un argumento pueden convertirse en funciones que toman sus argumentos uno a la vez (aplicación parcial) y devuelven otras funciones. Esta técnica de transformación se denomina *currying* y es una característica presente en los lenguajes funcionales modernos (e.g. Haskell).

Características

Existen numerosos lenguajes de PF, cada uno con implementaciones más o menos diferentes (e.g. LISP (1950), ISWIM (1960), FP (1970), ML (1970), Miranda (1980)). Además de estos, se destaca el lenguaje *Haskell*, un lenguaje funcional puro, con estrategia de evaluación perezosa que incluye muchas de las características implementadas en los lenguajes que le antecedieron. El desarrollo de Haskell comenzó en 1987, cuando un comité internacional de investigadores decidió unificar las principales características de los lenguajes funcionales en uno que sirviera de referencia y representara a este paradigma de programación. Después de 12 años de arduo trabajo en su diseño, en 1999 se publica una versión estable del lenguaje conocida como "Haskell 98 Report". Luego en 2003 se hace pública una nueva versión revisada bajo el nombre de "Haskell 98 Language and Libraries: The Revised Report".

Teniendo en cuenta la posición de Haskell como lenguaje funcional de referencia, en el curso se introducirán las principales características de la PF a través del estudio de este lenguaje. Para comenzar, se mencionan a continuación algunas de sus características, que serán tema de estudio durante esta y las próximas conferencias.

- **Programas claros y concisos**: los programas se expresan en forma de ecuaciones que definen funciones en un lenguaje muy cercano al lenguaje de las funciones matemáticas, y por tanto, más cercano al razonamiento lógico.
- **Sistema de tipos estático y fuerte (***statically-strongly-typed***)**: toda expresión del lenguaje tiene un tipo asociado. Antes de evaluar una expresión se comprueba si sus argumentos son del tipo especificado lo cual elimina los errores en tiempo de ejecución por incompatibilidades de tipo. Haskell incorpora un poderoso mecanismo de *inferencia de tipos* (*type inference*) para liberar al programador de su declaración. La inferencia de tipos también le permite hacer programas más generales. Si una función toma dos argumentos y los suma, no es necesario especificar el tipo de los argumentos, la función puede ser aplicada sobre cualquier par de valores que puedan actuar como números.
- **Polimorfismo**: los tipos y las funciones pueden ser polimórficas.
- **Funciones recursivas**: la recursividad es un poderoso instrumento de definición y un mecanismo básico e indispensable para definir funciones en los lenguajes funcionales.

- **Funciones de orden superior (***higher-order functions***)**: las funciones en los lenguajes funcionales son como las relaciones para los lenguajes de programación lógica y los objetos para los lenguajes de programación orientada a objetos. Son tratadas como *ciudadanos de primera clase*, lo cual significa que pueden almacenarse en estructuras de datos, pasarse como argumentos a otras funciones y devolver como resultado otras funciones.
- **Transparencia referencial**: las funciones denotan verdaderas funciones matemáticas, i.e., el resultado de una función depende únicamente de sus argumentos de entrada, propiedad que se conoce como transparencia referencial y conlleva a la ausencia de efectos colaterales.
- **Evaluación perezosa (***lazy-evaluation***)**: al evaluar una expresión, los argumentos de las funciones solo son evaluados si son necesarios para computar el valor de la expresión. Este tipo de estrategia de evaluación permite la computación con estructuras de datos potencialmente infinitas y la optimización de código.
- **Otras características**: correspondencia de patrones, comprensión de listas, funciones curried.

Definición de programas

Un *programa* en Haskell consiste de un conjunto de ecuaciones que definen funciones. Una función es una correspondencia que toma uno o más argumentos y produce un único resultado, similar al concepto de función matemática, y se define mediante una ecuación (o varias) compuesta por el nombre de la función, el nombre de cada uno de sus argumentos y un cuerpo, que especifica cómo se calcula el resultado a partir de los argumentos de entrada. De manera general, la definición de una función f se realiza mediante un conjunto de ecuaciones con el formato:

```
f <x1> <x2> ... <xn> = <expresion>
```

donde <x1> <x2> ... <xn> representan los argumentos de la función y <expresión> representa su cuerpo. El número n denota la cantidad de argumentos y se denomina aridad de la función. Si f fuese definida por más de una ecuación, entonces éstas deberían definirse juntas y con la misma aridad

Por ejemplo, la siguiente ecuación define la función doble que a cada número x que toma como argumento le asigna el valor x + x.

```
doble x = x + x
```

Las listas son una estructura de datos muy útil y muy empleada en los lenguajes funcionales. En Haskell, el símbolo ":" representa el constructor de listas y se utiliza en notación h : t, donde h representa a un elemento y t a una lista. Utilizando esta notación, el problema de concatenar dos listas en Haskell se resuelve definiendo una función concatena de dos argumentos (las listas que se desean concatenar), y que devuelve como resultado la concatenación de dichas listas. La definición se hace de manera recursiva empleando dos ecuaciones, una para el caso base y la otra para el caso recursivo.

```
concatena [] xs = xs
concatena (x:xs) ys = x : (concatena xs ys)
```

Un ejemplo más complejo implementa el algoritmo de ordenación *Quicksort* y evidencia características más avanzadas del lenguaje como la comprensión de listas.

```
qsort [] = []
qsort (x:xs) = qsort [a | a <- xs, a <= x] ++ [x] ++ qsort [b | b <- xs, b > x]
```

A primera vista parece ser que con esta implementación solo se pueden ordenar listas de números, sin embargo, la función es capaz de ordenar listas con valores de cualquier tipo que sean ordenables (instancias de la clase Ord). En Haskell todas las expresiones, incluyendo las funciones, tienen un tipo y el tipo de qsort es:

```
qsort :: Ord a => [a] -> [a]
```

Lo cual quiere decir que qsort es una función que recibe como argumento una lista con valores de cualquier tipo, denotado por a que representa una *variable de tipo*, que, siendo instancia de la clase de tipo Ord, da como resultado una lista de valores de tipo a.

En Haskell, los números y las cadenas de caracteres son tipos instancias de la clase Ord, por lo que puede usarse qsort para ordenar cualquier lista de números así como cualquier cadena (e.g. "zeadb"). Las cadenas no son más que listas de caracteres simples.

Ejecución de programas

Para ejecutar un programa en Haskell se necesita de un editor de textos y un compilador o intérprete de Haskell. La extensión generalmente predeterminada para Haskell es *.hs*. Un intérprete para Haskell ampliamente conocido es *Hugs*, viene por defecto instalado en *Linux* (aunque existen versiones para *Windows*) y puede ser iniciado en una terminal con el comando hugs. Hugs implementa casi todas las características del estándar Haskell 98 y su rendimiento es suficiente para la mayoría de las aplicaciones. Existen también varios compiladores para Haskell, entre ellos el más conocido es el *Glasgow Haskell Compiler* (GHC). Este compilador también provee un modo de uso interactivo que funciona de manera similar a Hugs y puede invocarse tecleando ghci en una terminal. Cuando se inicia cualquiera de los dos intérpretes hugs o ghci, lo primero que este hace, después de procesar las opciones, es cargar el módulo estándar **Prelude** el cual posee un conjunto de funciones predefinidas para el procesamiento de los tipos básicos predefinidos (e.g. números y listas). Los programas y fragmentos de código que se verán en los textos de conferencia han sido comprobados usando el intérprete Hugs.

Aplicación de funciones

Haskell es un lenguaje orientado a la evaluación de expresiones funcionales, o sea, a la aplicación de funciones a sus argumentos. Para aplicar una función a sus argumentos se escribe el nombre de la función y seguidamente, separado por espacios, sus argumentos.

Por ejemplo, la expresión f(a, b) + c ∗ d, que en matemática tiene el significado: sumar el resultado de evaluar la función f sobre los argumentos a y b con el resultado de multiplicar c y d, en Haskell se escribiría:

```
f a b + c ∗ d
```

Con respecto al resto de los operadores, la aplicación de funciones tiene la prioridad más alta. Por ejemplo, la expresión f a + b se interpreta como (f a) + b. Esto exige el uso de paréntesis para hacer explícita la evaluación de una expresión con el significado deseado. Por ejemplo, la expresión matemática f(x, g(y)) tendría que escribirse como f x (g y), pues sin paréntesis, o sea, f x g y, se entendería como la aplicación de f a tres argumentos, lo cual daría error si f fuera de aridad 2.

Evaluación de expresiones

Cuando se aplica una función a los argumentos de entrada, el resultado se obtiene sustituyendo los argumentos dentro del cuerpo de la función. Este proceso puede producir inmediatamente un resultado que no se puede simplificar, como un número o un carácter. Sin embargo, lo más común es que el resultado de la sustitución sea una expresión que contenga otras aplicaciones de funciones, las cuales deben ser procesadas de igual manera hasta producir el resultado final.

El proceso de evaluación consiste en tomar una expresión e ir reduciéndola aplicando las definiciones de funciones (introducidas por el programador o predefinidas) hasta que no pueda reducirse más. La expresión resultante se denomina representación canónica y es mostrada al usuario.

Por ejemplo, el resultado de aplicar la función doble al número 3 se haría de la siguiente manera:

```
doble 3   { aplicando doble }
= 3 + 3   { aplicando + }
= 6
```

De manera similar, el resultado de la expresión doble (doble 2), donde la función doble se aplica dos veces se calcula de la siguiente forma:

```
doble (doble 2)    { aplicando doble interno }
= doble (2 + 2)    { aplicando + }
= doble 4          { aplicando doble }
= 4 + 4            { aplicando + }
= 8
```

El mismo resultado se obtiene si se comienza aplicando la función doble externa:

```
doble (doble 2)        { aplicando doble externo }
= doble 2 + doble 2    { aplicando primer doble }
= (2 + 2) + doble 2    { aplicando primer + }
= 4 + doble 2          { aplicando doble }
= 4 + (2 + 2)          { aplicando segundo + }
= 4 + 4                { aplicando + }
= 8
```

Sin embargo, en esta evaluación se realizaron dos reducciones más que en la anterior. Esto se debe a que en el primer paso de reducción la expresión doble 2 se duplica y luego debe reducirse dos veces. De manera general, el orden en que las funciones son aplicadas en una evaluación no afecta el resultado final, sin embargo, si puede afectar el número de pasos de reducción, y por tanto, el tiempo que requiere terminar una evaluación. Existen diferentes estrategias de evaluación, por ejemplo, la *evaluación perezosa* (*lazy evaluation*) y la impaciente (*eager evaluation*). En una evaluación impaciente los argumentos de una función se evalúan antes de ejecutar la función, mientras que una evaluación perezosa estos se evalúan en la medida que sean necesarios para computar el valor de la función. Más adelante se profundizará sobre las características de cada una de estas estrategias.

Descripción del dominio: *tipos* y *clases de tipos*

Una de las características de los lenguajes funcionales es la necesidad de emplear recursos expresivos para fijar el perfil de la función, i.e., su dominio y su rango. En PF la descripción de dominios conduce a la idea de *tipos de datos*. Los tipos de datos se introducen en programación para detectar y evitar errores y ayudar a definir estructuras de datos.

Haskell es estático y fuertemente tipado (*statically-strongly-typed*). Toda expresión del lenguaje tiene un tipo asociado, que denota el conjunto de valores que puede tomar y, además, el tipo de cada expresión se calcula previo a su evaluación para realizar una comprobación de tipos y asegurarse de que no se producirán errores durante su evaluación por incompatibilidades de tipo. También incorpora un mecanismo automático de inferencia de tipos para liberar al programador de la necesidad de declararlos ya que son deducidos por el intérprete.

Tipo

Un *tipo* es una colección de valores relacionados. Por ejemplo, el tipo Bool contiene los valores lógicos True y False, mientras que el tipo Int -> Int contiene todas las funciones que aplicadas a números enteros dan como resultado un número entero.

La notación exp :: T significa que exp es una expresión de tipo T y se lee exp "tiene tipo" T. Por ejemplo:

```
True :: Bool                3 :: Int
'a' :: Char                 length "haskell" :: Int
```

Haskell posee un conjunto de tipos predefinidos y brinda facilidades para la declaración de nuevos tipos. A continuación se describen los tipos predefinidos y se mencionan algunas de las funciones definidas en el estándar Prelude para cada uno.

Tipos básicos

Bool contiene los valores lógicos True y False. Funciones definidas: &&,||,not.

Char contiene todos los caracteres simples disponibles en un teclado estándar, por ejemplo, 'a', 'Z', '5', '_', '.' etc. Los caracteres especiales deben ser escritos utilizando el símbolo de escape \ (*backslash*), por ejemplo, \\,\',\",\n.

String contiene todas las secuencias de caracteres, por ejemplo, "abc", "1+2=3", la cadena vacía "". El tipo String se define como una abreviación de una lista de caracteres (type String = [Char]).

Int contiene los números enteros de precisión fija. En algunos intérpretes de Haskell como *Hugs* solo se consideran valores de tipo Int a los números enteros en el rango $[-2^{31};2^{31}-1]$. Esto hace, por ejemplo, que la evaluación de 2^31 :: Int que fuerza el resultado de 2^31 a ser de tipo Int produzca como respuesta un número negativo, lo cual es incorrecto. Funciones definidas: +, *, -, ^, negate, div, rem, mod, odd, even, gcd, lcm, abs, signum.

Integer contiene todos los números enteros (de precisión arbitraria). La evaluación de 2^31 :: Integer produce el resultado correcto esperado.

Float contiene números flotantes de precisión simple: -12.34, 1.0, 4.13165. Se puede utilizar notación científica para representarlos, e.g. 1.0e3 equivale a 1000,0 y 5.0e-2 equivale a 0,05. Funciones definidas: pi, exp, log, sqrt, sin, cos, tan, asin, acos, atan.

Double contiene números flotantes de precisión doble.

Note que un número puede tener más de un tipo numérico, o sea, 3 :: Int, 3 :: Integer, y 3 :: Float son tipos válidos para el número 3. En este sentido, surge una pregunta interesante, ¿qué tipo se le asigna a un número durante la inferencia de tipos? Si en un intéprete de Haskell se escribe la expresión :type 3 se obtendrá como respuesta 3 :: Num a => a. Esto significa, a grandes rasgos, que el tipo del número 3 puede ser cualquier tipo que sea instancia de la clase de tipo Num, la cual define un conjunto de operaciones básicas sobre números.

A partir de los tipos básicos, Haskell introduce dos tipos de estructuras de datos fundamentales, las *listas* y las *tuplas*.

Listas

Las listas fueron introducidas en el lenguaje LISP y constituyen uno de los grandes aportes de la PF a la programación, dada su extraordinaria capacidad para representar cualquier tipo de dato e información a partir de los tipos básicos.

Una *lista* es una secuencia (potencialmente infinita) de elementos, **todos del mismo tipo**, encerrados entre corchetes y separados por comas. Se escribe [T] para declarar el tipo de todas las listas cuyos elementos son del tipo T. Por ejemplo:

```
[False, True, False ] :: [Bool]
['a', 'b', 'c', 'd'] :: [Char]
["Hola", "Haskell"] :: [String]
[[1,3,5], [2,4,6], []] :: [[Int]]
```

El símbolo : representa el operador de construcción de lista y se utiliza en notación infija: h : t donde h es un elemento y t es una lista. El resultado de esta operación es una lista que tiene como primer elemento a h y como resto a la lista t. Por ejemplo, las expresiones [1]:[2]:[[3]] y 'h':'o':"la" dan como resultado [[1],[2],[3]] y "hola" respectivamente. La evaluación de la segunda expresión es posible y no da error ya que las cadenas son listas de caracteres simples, luego todas las funciones del estándar Prelude para listas pueden ser utilizadas también con cadenas.

Se puede observar en la manera de construir una lista que el tipo de datos lista es recursivo. La recursividad se advierte en cualquier lista no vacía: [e1,…,ek] = e1 : (e2 : (… (ek : []))) donde los elementos ei son todos del mismo tipo.

Funciones predefinidas sobre listas: head, tail, init, length, null, reverse, take, drop, maximum, minimum, sum, product, elem, etc. Los operadores ++, !! representan las operaciones de concatenación e indexación, respectivamente. Se utilizan en notación infija, por ejemplo, la expresión ([1] ++ [2, 3]) !! 2 da como resultado el valor 3. Las listas en Haskell se enumeran a partir de 0.

Tuplas

Una *tupla* es una secuencia finita de componentes, posiblemente de tipos diferentes, encerrados entre paréntesis y separados por coma. Se escribe (T1,T2,…,Tn) para declarar el tipo de todas las tuplas cuyas componentes son del tipo Ti para toda *i* en el rango 1,*…*,*n*. Por ejemplo:

```
(False, True) :: (Bool, Bool)
("Ana", True, 32) :: (String, Bool, Int)
('a', (False, 'b')) :: (Char , (Bool , Char))
(['a', 'b'], [False, True ]) :: ([Char], [Bool])
```

Funciones predefinidas sobre tuplas: fst, snd, zip, unzip.

A diferencia de las listas, que son homogéneas, las tuplas pueden combinar valores de diferentes tipos, sin embargo, deben tener una aridad finita. Las listas pueden ser infinitas.

En determinadas aplicaciones podría ser útil trabajar con una tupla especial con 0 elementos denominada tipo unidad (*unit*). El tipo unidad se escribe como () y tiene un único elemento que es también (). Se puede ver como algo similar al void de C# y usarse, por ejemplo, para especificar que una función no devuelve ningún valor.

Funciones

Se escribe T1 -> T2 para declarar el tipo de todas las funciones que toman argumentos de tipo T1 y producen como resultado valores de tipo T2. Por ejemplo:

```
not :: Bool -> Bool
doble :: Int -> Int
concatena :: [a] -> [a] -> [ a ]
```

En los dos primeros ejemplos se han declarado tipos de funciones que toman como argumentos y retornan como resultados valores de tipos básicos (e.g Bool, Int). En la declaración del tipo de la función concatena, sin embargo, se ha hecho uso del símbolo a. Este símbolo representa una *variable de tipo* y significa que la función concatena puede aplicarse a listas de diferentes tipos (e.g. listas de enteros, de caracteres, listas de listas, etc).

Esta facultad se denomina *polimorfismo* y permite la creación de funciones y tipos de datos genéricos, de forma tal que pueden manejar diferentes valores de manera idéntica independientemente de su tipo. En un lenguaje sin polimorfismo sería necesario definir una función concatena para cada tipo de lista que se necesitase. El polimorfismo permite una mayor reutilización de código ya que no es necesario repetir algoritmos para estructuras similares.

En particular, este tipo de polimorfismo se denomina *polimorfismo paramétrico*. Fue formalmente desarrollado en una extensión del cálculo lambda (denominada cálculo lambda polimórfico) y constituye un rasgo característico y muy apreciado de la PF. Programación genérica es como suele denominarse a esta característica en la programación orientada a objetos.

Tipos polimórficos

Los tipos que contienen al menos una variable de tipo en su declaración son *tipos polimórficos*. Las variables de tipo se denotan por letras en minúscula (e.g. a,b,c). De acuerdo a su declaración paramétrica, las listas en Haskell son tipos polimórficos:

```
data [a] = [] |(a:[a])
```

Los tipos polimórficos definen familias de tipos. Por ejemplo: [a] denota la familia de listas de tipo a:

[Int], [Char], [(Int,Int)]...

El tipo [a] -> [a] -> [a] es también un tipo polimórfico y define la familia de funciones que toman como argumentos dos listas de tipo a y devuelven una lista de tipo a.

Funciones polimórficas

Una *función polimórfica* es aquella que no está definida para un tipo en particular sino que puede ser aplicada a valores de diferentes tipos. En Haskell se reconoce una función polimórfica cuando en la definición de su tipo aparece al menos una variable de tipo. La función concatena definida anteriormente es una función polimórfica. El siguiente ejemplo muestra la función fst y su declaración de tipo, ambos polimórficos:

```
fst :: (a, b) -> a
fst (x,_) = x
```

La declaración de la función fst de esta forma hace que luego pueda ser aplicada sobre cualquier tupla de dos elementos independientemente del tipo que estos sean.

Muchas de las funciones predefinidas en Haskell son polimórficas. Por ejemplo:

```
length :: [a] -> Int                head :: [a] -> a
take :: Int -> [a] -> [a]           zip :: [a] -> [b] -> [( a,b )]
```

Tipos con sobrecarga

El operador aritmético + calcula la suma de dos números cualesquiera del mismo tipo numérico. Por ejemplo, puede usarse para calcular la suma de dos números enteros así como la de dos números flotantes:

```
Hugs> 1+2
3
Hugs> 1.1 + 2.2
3.3
```

Que el operador + pueda ser aplicado a números de cualquier tipo numérico se hace explícito en su declaración de tipo mediante la inclusión de una *restricción de clase*. Las restricciones de clase se escriben en la forma C a, donde C es el nombre de una *clase de tipo* y a es una variable de tipo. Por ejemplo, el tipo de + es el siguiente:

```
(+) :: Num a => a -> a -> a
```

Esto significa que para cualquier tipo a, si a es una instancia de la clase Num, la función (+) tiene tipo a -> a -> a. Los tipos que contienen una o más restricciones de clase son llamados *tipos con sobrecarga*. Por ejemplo, Num a => a -> a -> a es un tipo con sobrecarga y (+) es una función con sobrecarga. Muchas de las funciones predefinidas en Haskell son sobrecargadas. Por ejemplo:

```
(+) :: Num a => a -> a -> a
(−) :: Num a => a -> a -> a
(∗) :: Num a => a -> a -> a
negate :: Num a => a -> a
abs :: Num a => a -> a
signum :: Num a => a -> a
```

Los números son también tipos con sobrecargas. Por ejemplo, 3 :: Num a => a significa que para cualquier tipo numérico a, el número 3 es de tipo a.

Clases de tipos

Una *clase de tipo* es una especie de interfaz que define algún comportamiento. Si un tipo es instancia de una clase de tipo, eso significa que este implementa el comportamiento que la clase describe. De manera general, una clase de tipo puede verse como una colección de tipos que soportan ciertas operaciones con sobrecarga. La definición de clases de tipos es una característica muy valorada de Haskell. Entre sus utilidades está la posibilidad de definir funciones genéricas que toman como argumentos cualquier tipo que describa el comportamiento específico que declara la clase para la cual se definió la función.

Haskell contiene un número de clases predefinidas en el lenguaje, de las cuales las más utilizadas se describen a continuación:

Eq para tipos cuyos valores pueden ser comparados respecto a un criterio de igualdad o desigualdad. Define las siguientes funciones:

```
class Eq a where
    (==), (/=) :: a -> a -> Bool
    x /= y = not (x == y)
    x == y = not (x /= y)
```

Ord para tipos que pueden ser ordenables. Define las siguientes funciones:

```
class Ord a where
    (<) :: a -> a -> Bool                (>) :: a -> a -> Bool
    (<=) :: a -> a -> Bool               (>=) :: a -> a -> Bool
    min :: a -> a -> a                   max :: a -> a -> a
```

Los tipos básicos Bool, Char, String, Int, Integer y Float son instancias de la clase Ord, así como las listas y tuplas cuyos elementos y componentes sean instancias de la clase Ord.

Num para tipos numéricos. Define las siguientes funciones:

```
class Num a where
    (+) :: a -> a -> a                   (−) :: a -> a -> a
    (∗) :: a -> a -> a                   negate :: a -> a
    abs :: a -> a                        signum :: a -> a
```

Los tipos básicos Int, Integer y Float son instancias de la clase Num.

Show para tipos cuyos valores pueden convertirse a una cadena de caracteres. Define una sola función:

```
class Show a where
    show :: a -> String
```

Los tipos básicos Bool, Char, String, Int, Integer y Float son instancias de la clase Show, así como las listas y tuplas cuyos elementos y componentes sean instancias de esta clase.

Read para tipos cuyos valores pueden obtenerse a partir de una cadena de caracteres. Define una sola función:

```
class Read a where
    read :: String -> a
```

Los tipos básicos Bool, Char, String, Int, Integer y Float son instancias de la clase Read, así como las listas y tuplas cuyos elementos y componentes sean instancias de esta clase. Ejemplos de cómo usar la función read:

```
Hugs> read "123" :: Int
123
Hugs> read "[True, False]" :: [Bool]
[True, False]
```

Existen otras como la clase Integral, para tipos numéricos que soportan la división entera (div) y el resto de la división entera (mod), y la clase Fractional también para tipos numéricos pero que soportan la división fraccionaria (/) y el recíproco (recip).

Además de las clases que vienen predefinidas en el lenguaje, el usuario puede definir sus propias clases así como declarar nuevos tipos como instancias de las clases ya definidas. La sintaxis que Haskell proporciona para esto es muy sencilla y se estudiará en próximas conferencias.

Definición de funciones

En esta sección se describen diferentes formas de definir una función y las facilidades que proporciona el lenguaje para ello.

Utilizando funciones definidas

La forma más simple de definir una función es mediante la utilización de funciones previamente definidas. Por ejemplo, utilizando la función doble se puede definir una función que calcula el cuádruplo de un número:

```
cuadruplo :: Num a => a -> a
cuadruplo x = doble x + doble x
```

Expresiones condicionales

Haskell provee diferentes maneras de definir funciones que seleccionan entre un conjunto de posibles resultados. La forma más simple es con la expresión condicional if then else. Por ejemplo, la función abs que retorna el valor absoluto de un número puede definirse como:

```
abs :: Num a => a -> a
abs n = if n >= 0 then n else −n
```

Las expresiones condicionales pueden anidarse. Por ejemplo, la función signum que retorna el signo de un número puede definirse como:

```
signum :: Num a => a -> a
signum n = if n < 0 then −1 else if n == 0 then 0 else 1
```

Guardas

Como alternativa al uso de expresiones condicionales anidadas, las funciones con múltiples expresiones if then else pueden definirse usando ecuaciones con *guardas*. Las guardas son expresiones lógicas que se utilizan para escoger de entre una secuencia de posibles resultados del mismo tipo. Si la primera guarda es verdadera, entonces se retorna el primer resultado; si no, se verifica si la segunda guarda es verdadera en cuyo caso se retorna el segundo resultado y así sucesivamente. La sintaxis de una ecuación con guardas es la siguiente:

```
f <x1> <x2> ... <xn> | <condicion1> = < expresion1>
                     | <condicion2> = < expresion2>
                     ...
                     | <condicionk> = < expresionk>
```

Por ejemplo, las funciones abs y signum se pueden definir usando guardas de la siguiente forma:

```
abs n | n >= 0 = n
      | otherwise = −n

signum n | n < 0 = −1
         | n == 0 = 0
         | otherwise = 1
```

El símbolo "|" se lee "tal que" y la guarda otherwise se define en el Prelude como otherwise = True. El uso de otherwise para terminar una secuencia de guardas no es necesario, pero resulta conveniente para el manejo de todos los casos posibles de entrada, incluyendo aquellos para los que ninguna de las guardas de la secuencia se hace verdadera, lo cual produciría un error.

Correspondencia de patrones (*pattern matching*)

Una función también puede definirse mediante un conjunto de ecuaciones utilizando la técnica de *correspondencia de patrones*, en la que se usa una secuencia de expresiones sintácticas llamadas *patrones* para seleccionar de entre una secuencia de resultados del mismo tipo. La definición de una función de esta forma es la siguiente:

```
f <patron1> = <expresion1>
f <patron2> = <expresion2>
...
f <patronk> = <expresionk>
```

donde en cada ecuación se especifica un patrón determinado así como la expresión que debe ser evaluada si se verifica la presencia de dicho patrón en los argumentos de entrada de la función.

Cuando se aplica una función definida de esta forma a sus argumentos de entrada, se realiza un proceso en el que se verifica si existe una correspondencia entre los argumentos pasados a la función y los patrones definidos en cada ecuación. De esta forma se determina cuál es la ecuación que se debe seleccionar y, por tanto, qué expresión evaluar. Esta comprobación se realiza en el orden en que fueron definidos los patrones: primero con patron1, luego con patron2 y así sucesivamente hasta que algún patrón coincida, en cuyo caso la expresión correspondiente es evaluada y devuelta.

La correspondencia de patrones puede hacerse sobre variables y cualquier tipo de dato definido: números, caracteres, listas, tuplas, etc. A continuación se muestran algunos ejemplos de funciones definidas con correspondencia de patrones.

Patrones sobre constantes numéricas y variables

Se define la función factorial:

```
factorial :: Num a => a -> a
factorial 0 = 1
factorial n = n * factorial (n-1)
```

Patrones sobre valores de tipo Bool

En este ejemplo se define la función miAnd, con un comportamiento similar al del operador && para valores booleanos, haciendo explícito todas las posibles entradas que se le puede pasar a la función:

```
miAnd :: Bool -> Bool -> Bool
miAnd True True = True
miAnd True False = False
miAnd False True = False
miAnd False False = False
```

Patrón *anónimo*

La definición anterior puede simplificarse, combinando las últimas tres ecuaciones en una que retorna False independientemente de los valores que tengan los argumentos. Esto puede hacerse usando el patrón anónimo _ que se ajusta a cualquier valor:

```
miAnd True True = True
miAnd _ _ = False
```

Patrones sobre listas

Se define la función sumaLista, que devuelve la suma de los elementos de una lista (similar a la función sum definida en el estándar Prelude). Se utiliza el patrón de lista (x:xs) para referirse a una lista que tiene al menos un elemento.

```
sumaLista :: Num a => [a] -> a
sumaLista [] = 0
sumaLista (x:xs) = x + sumaLista xs
```

El patrón (x:xs) puede generalizarse para denotar listas con al menos k elementos. La siguiente función imprime un texto de acuerdo a la clasificación que haga sobre la lista atendiendo a su longitud:

```
clasifica :: Show a => [a] -> String
clasifica [] = "La lista está vacía"
clasifica [x] = "La lista tiene un elemento: " ++ show x
clasifica [x, y] = "La lista tiene dos elementos: " ++ show x ++ " y " ++ show y
clasifica (x:y:_) = "La lista es extensa. Los primeros dos elementos son: "++ show x ++ " y " ++ show y
```

Patrones sobre tuplas

La función sumaVectores, que toma dos vectores de 2 dimensiones como argumentos en forma de tuplas y devuelve el vector que resulta de sumar sus componentes separadamente, se puede definir de la siguiente forma:

```
sumaVectores :: (Num a, Num b) => (b,a) -> (b,a) -> (b,a)
sumaVectores a b = (fst a + fst b, snd a + snd b)
```

Sin embargo, usando la correspondencia de patrones sobre tuplas, esta puede definirse de una forma más elegante:

```
sumaVectores (x1,y1) (x2,y2) = (x1 + x2, y1 + y2)
```

Patrones con nombre

Es posible dar nombre a todo un patrón poniendo un nombre y el símbolo @ delante de este, por ejemplo, xs@(x:y:ys). Este patrón es equivalente a (x:y:ys), pero permite acceder rápidamente a toda la lista a través de su nombre xs en lugar de repetir en el cuerpo de la función el patrón (x:y:ys). Un ejemplo de su uso:

```
elemento :: Show a => [a] -> String
elemento [] = "Lista vacía"
elemento lista@(x:xs) = "El primer elemento de " ++ show lista ++ " es " ++ show x
```

Patrones *n + k*

Un caso especial de los patrones con enteros, que en ocasiones puede ser útil, es el patrón de la forma n+k, donde n es una variable y k>0 es una constante, ambas enteras. Por ejemplo, la función fibonacci que determina el *n*-ésimo número en la secuencia de Fibonacci puede definirse con este patrón de la siguiente forma:

```
fibonacci :: (Integral a, Num b) => a -> b
fibonacci 0 = 1
fibonacci 1 = 1
fibonacci (n+2) = fibonacci n + fibonacci (n+1)
```

Bloques *where*

En la declaración de una función se pueden incluir definiciones locales para variables que podrían encontrarse en guardas o en la parte derecha de una ecuación. Para ello pueden usarse la construcción where que se expresa como:

```
where <var1> = < expresion1>
      <var2> = < expresion2>
      ...
      <vark> = < expresionk >
```

o de manera alternativa, en una línea, como:

```
where (<var1>,<var2>,...,<vark>) = (<expresion1>,<expresion2>,...,<expresionk>)
```

El algoritmo qsort definido anteriormente

```
qsort [] = []

qsort (x:xs) = qsort [a | a <- xs, a <= x] ++ [x] ++ qsort [b | b <- xs, b > x]
```

puede definirse de una manera más legible usando la construcción where de la siguiente forma:

```
qsort :: Ord a => [a] -> [a]
qsort [] = []
qsort (x:xs) = qsort menores ++ [x] ++ qsort mayores
    where menores = [a | a <- xs, a <= x]
          mayores = [b | b <- xs, b > x]
```

La palabra clave where se pone en el cuerpo de la función luego de la expresión principal y se utiliza para enlazar dos nombres, menores y mayores, a expresiones que son evaluadas antes de evaluar la expresión principal de la función.

Una de las ventajas del uso del where es que se pueden cambiar las expresiones que se enlazan a un nombre sin modificar el cuerpo de la función. También se utiliza para dar nombre a una expresión que se usa muchas veces en el cuerpo de la función para que esta se evalúe solo una vez.

Además de constantes, en las construcciones where también se pueden definir funciones. Por ejemplo, la siguiente función toma una lista de pares (peso, altura) y retorna una lista con el BMI correspondiente a cada par. La función bmi que calcula el BMI para un par en particular se define dentro del bloque where.

```
calcbmis :: Fractional a => [(a,a)] -> [a]
calcbmis xs = [bmi p a | (p,a) <- xs]
    where bmi p a = p / a^2
```

Expresión *let...in*

Las expresiones let…in son en cierto grado muy similares a las construcciones where. Estas permiten también definir variables, pero a diferencia de las otras son locales a la expresión que la constituye luego no son visibles por ejemplo en las guardas. La sintaxis de let…in es la siguiente:

```
let <var1> = <expresion1>
    <var2> = <expresion2>
    ...
    <vark> = <expresionk>
in < expresion >
```

o de manera alternativa se puede expresar en una línea como:

```
let <var1> = <expresion1>; ... ; <vark> = <expresionk> in <expresion>
```

Un ejemplo de su uso se muestra en la siguiente definición:

```
area_cilindro :: Float a => a -> a -> a
area_cilindro r h = let lateral = 2 * pi * r * h
                        base = pi * r ^ 2
                    in lateral + 2 * base
```

La diferencia fundamental entre let…in y la construcción where es que la primera constituye en sí misma una expresión. Esto significa que se puede usar let…in donde quiera que sea válido poner una expresión.

En las listas por comprensión también pueden usarse. Por ejemplo, la siguiente es una definición alternativa para la función calcbmis definida anteriormente:

```
calcbmis :: Fractional a => [(a,a)] -> [a]
calcbmis xs = [bmi | (p,a) <- xs, let bmi = p / a ^ 2]
```

En este caso se omite la parte correspondiente al "in" ya que la visibilidad de la variable bmi se asume que alcanza la parte delante del símbolo | y los predicados definidos después de la expresión let. Por ejemplo, la siguiente versión de la función calcbmis retorna la lista de los BMI clasificados como "normal":

```
calcbmis xs = [bmi |(p,a) <- xs, let bmi = p / a^2, bmi > 18.5 && bmi <= 25.0]
```

Expresión *case...of*

La expresión case…of puede ser utilizada para evaluar una expresión y, dependiendo del resultado, devolver uno de los posibles valores definidos. Su sintaxis es la siguiente:

```
case <expresion> of <patron1> -> <resultado1>
                    <patron2> -> <resultado2>
                    ...
                    <patronk> -> <resultadok>
```

Ejemplo:

```
paridad :: (Num a, Integral b, Num c) => ((b -> b -> b) -> a -> c) -> [Char]
paridad x = case (x mod 2) of 0 -> "par"
                              1 -> "impar"
```

```
factorial2 :: Num a => a -> a
factorial n = case n of 0 -> 1
                        _ -> n * factorial (n-1)
```

Funciones recursivas

La recursión es el mecanismo básico para realizar operaciones repetitivas en los lenguajes funcionales. Algunos de los ejemplos anteriores (e.g. qsort, fibonacci) corresponden a *funciones recursivas*, donde sus cuerpos se han expresado en términos de ellas mismas. Por ejemplo:

Función que invierte el orden de los elementos de una lista:

```
invierte :: [a] -> [a]
invierte [] = []
invierte (x:xs) = invierte xs ++ [x]
```

Función que inserta un elemento en una lista ordenada:

```
inserta_ord :: Ord a => a -> [a] -> [a]
inserta_ord x [] = [x]
inserta_ord x (y:ys) | x <= y = x:y:ys
                     | otherwise = y:inserta_ord x ys
```

Función que ordena los elementos de una lista usando la función inserta_ord:

```
ordena_ins :: Ord a => [a] -> [a]
ordena_ins [] = []
ordena_ins (x:xs) = inserta_ord x (ordena_ins xs)
```

Función que toma dos listas y devuelve una lista de tuplas:

```
pares :: [a] -> [b] -> [(a,b)]
pares _ [] = []
pares [] _ = []
pares (x:xs) (y:ys) = (x,y) : pares xs ys
```

Función que devuelve una lista sin sus primeros *n* elementos:

```
elimina :: Int -> [a] -> [a]
elimina 0 xs = xs
elimina (n + 1) [] = []
elimina (n + 1) (_:xs) = elimina n xs
```

Un ejemplo de *recursión mutua*, en la que se definen dos funciones, cada una en términos de la otra:

```
par :: Int -> Bool
par 0 = True
par (n + 1) = impar n
```

```
impar :: Int -> Bool
impar 0 = False
impar (n + 1) = par n
```